FACULDADE DE TECNOLOGIA DE SOROCABA
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

KARINA DE MORAES GARCIA
NICOLLY LEMOS DA SILVA
RAFAELA MANSANO FERNANDES
RAFAELA SILVA RIGANTI

IMPLANTAÇÃO DO DRE SYSTEM NA AWS:

Práticas de produção na AWS Academy

SOROCABA
2026

KARINA DE MORAES GARCIA
NICOLLY LEMOS DA SILVA
RAFAELA MANSANO FERNANDES
RAFAELA SILVA RIGANTI

IMPLANTAÇÃO DO DRE SYSTEM NA AWS:

Práticas de produção na AWS Academy

Trabalho apresentado à disciplina de Computação em Nuvem do curso de Tecnologia em Análise e Desenvolvimento de Sistemas da Faculdade de Tecnologia de Sorocaba, como requisito parcial para avaliação, sobre a implantação do sistema do Projeto Integrador II na AWS Academy.

Professor: Glauco

SOROCABA
2026

SUMÁRIO

1 INTRODUÇÃO....3
2 ARQUITETURA....3
2.1 JUSTIFICATIVA DOS SERVIÇOS....4
2.2 ORGANIZAÇÃO DOS RECURSOS....5
3 IMPLANTAÇÃO E INFRASTRUCTURE AS CODE....5
4 CI/CD COM GITHUB ACTIONS....6
4.1 INTEGRAÇÃO CONTÍNUA (CI)....6
4.2 ENTREGA CONTÍNUA (CD)....6
5 SEGURANÇA....7
5.1 IAM E LEAST PRIVILEGE....7
6 LOGS E MONITORAMENTO....8
7 BACKUP E RECUPERAÇÃO....9
7.1 RESTORE TESTADO....10
8 AMBIENTES....11
9 ROLLBACK....12
10 ESCALABILIDADE, TESTE DE CARGA E CUSTOS....12
10.1 CRESCIMENTO DA APLICAÇÃO....12
10.2 TESTE DE CARGA....13
10.3 CUSTOS....13
11 AWS ACADEMY × PRODUÇÃO....14
11.1 CDN E HTTPS....14
11.2 IAM....14
11.3 DEPLOY....15
11.4 DISPONIBILIDADE....15
11.5 AMBIENTES....15
11.6 BACKUP....15
11.7 UPLOADS....16
12 CONCLUSÃO....16
REFERÊNCIAS....17

1 INTRODUÇÃO

O **DRE System** (OAF Controle Financeiro) é o sistema do nosso Projeto Integrador II: um controle financeiro web em PHP 8 e MySQL que gera o DRE, o fluxo de caixa, o calendário de contas e guarda as notas fiscais ligadas aos lançamentos. Até aqui ele rodava no XAMPP, no computador de cada integrante.

Neste trabalho levamos o sistema para a AWS Academy. A ideia não foi só deixá-lo "no ar", mas montar a implantação do jeito que se faz em produção: HTTPS, senha fora do código, deploy automático, logs centralizados, alarmes, backup com restore testado, rollback e uma análise honesta de custo e de crescimento. Quando o laboratório não permitiu alguma prática, registramos a limitação e explicamos como ficaria numa conta AWS real.

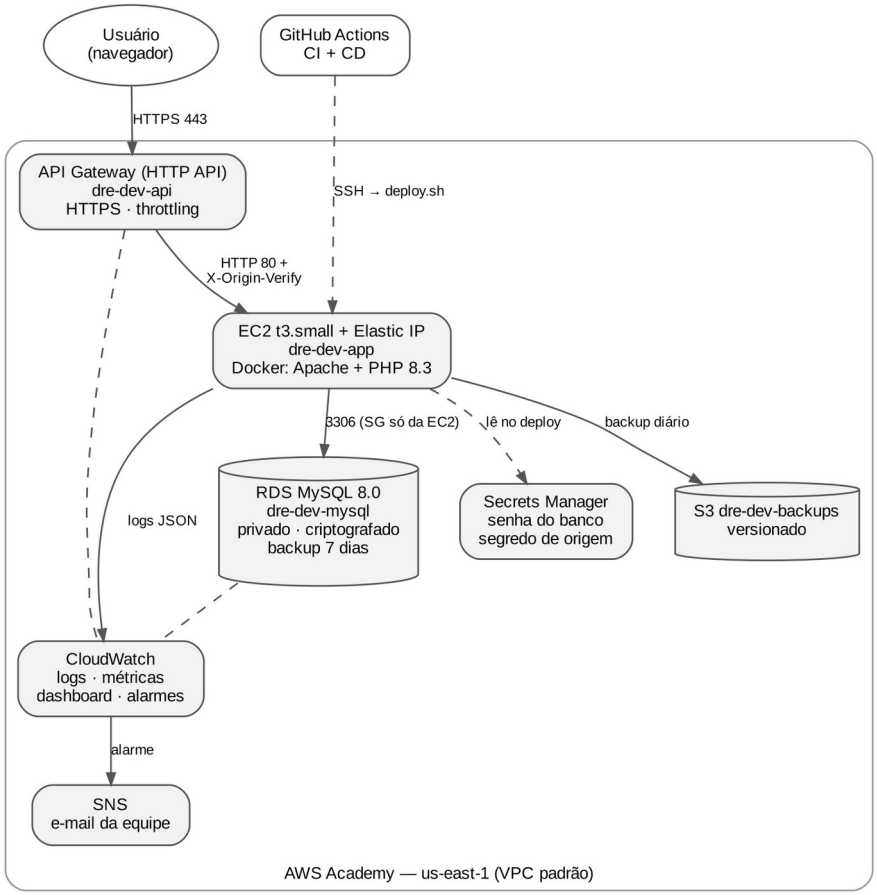
Toda a infraestrutura está descrita em código num único arquivo do CloudFormation, então ela pode ser apagada e recriada igual a qualquer momento.

Repositório: github.com/nicollylemos/DRE
Aplicação: 3udlpk22p9.execute-api.us-east-1.amazonaws.com

2 ARQUITETURA

O usuário acessa o sistema por HTTPS no API Gateway, que repassa a requisição para a EC2 onde a aplicação roda em Docker. O banco fica no RDS, sem IP público. A senha do banco é gerada e guardada no Secrets Manager, os backups vão para o S3 e tudo que acontece aparece no CloudWatch, que avisa a equipe por e-mail via SNS quando algo sai do normal. O GitHub Actions testa cada alteração e publica as versões novas na EC2.

Figura 1 – Arquitetura do DRE System na AWS



Fonte: elaborado pelos autores (2026).

2.1 JUSTIFICATIVA DOS SERVIÇOS

Quadro 1 – Justificativa dos serviços

EC2 + Docker	• A aplicação é PHP com Apache, sessões em arquivo e uploads em disco. Uma máquina virtual roda isso sem reescrever o sistema. • O Docker deixa a imagem igual no CI e na EC2 e guarda as versões anteriores para rollback rápido. Lambda exigiria reescrever o sistema para serverless.
RDS MySQL	• O sistema é relacional (lançamentos, categorias, caixas e parceiros ligados entre si, relatórios com JOIN e GROUP BY), então um banco NoSQL não serve. • O RDS cuida de patch, backup automático, restore em ponto no tempo, criptografia e crescimento do disco.
API Gateway	• Entrega um endereço HTTPS da própria AWS sem precisar de domínio, com limite de requisições (throttling) e métricas de tráfego, erros e latência. • O CloudFront seria a primeira escolha, mas o laboratório bloqueia a criação de distribuições (ver seção 11).

Secrets Manager	• A senha do banco é gerada pela AWS e nunca passa por Git, template ou tela. A EC2 lê o segredo na hora do deploy usando a role da instância, sem Access Key.
S3	• Destino barato e durável dos backups: dumps do MySQL e cópia dos uploads, comprovantes e notas. O versionamento protege contra arquivo apagado ou sobrescrito por engano.
CloudWatch + SNS	• Logs centralizados (o Docker envia direto), métricas de EC2, RDS e API Gateway, métricas tiradas dos próprios logs, dashboard e alarmes com aviso por e-mail.
Elastic IP	• O laboratório desliga a EC2 no fim da sessão. O IP fixo mantém o destino do API Gateway e o endereço usado pelo deploy sempre iguais.

Fonte: elaborado pelos autores (2026).

2.2 ORGANIZAÇÃO DOS RECURSOS

Todos os recursos seguem o padrão `projeto-ambiente-recurso` (`dre-dev-app`, `dre-dev-mysql`, `dre-dev-api`, `/dre/dev/app`) e recebem as tags `Project=DRE`, `Environment` e `Team`. Assim fica fácil filtrar custo e saber a que ambiente cada coisa pertence.

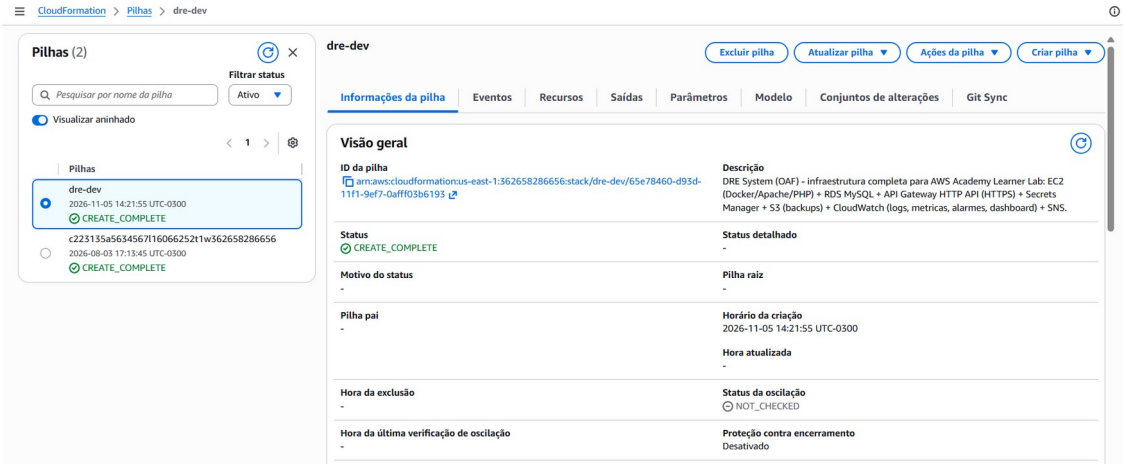
3 IMPLANTAÇÃO E INFRASTRUCTURE AS CODE

Quadro 2 – Implantação e Infrastructure as Code

Como funciona	• O arquivo `infra/cloudformation.yaml` cria a stack `dre-dev` inteira: Security Groups, RDS, Secrets Manager, S3, EC2 com Elastic IP, API Gateway, log group, filtros de métrica, alarmes, dashboard e tópico SNS. • No primeiro boot a EC2 instala o Docker, clona o repositório e roda `scripts/deploy.sh`. A stack só termina como CREATE_COMPLETE depois que o `/health` responde UP.
Benefícios	• A infraestrutura pode ser apagada e recriada igual, sem depender de cliques no console. • Se algo falha na criação, o próprio CloudFormation desfaz tudo — foi o que aconteceu na primeira tentativa, com o CloudFront bloqueado.
Onde está	• `infra/`, `scripts/`, `Dockerfile` e `docker-compose.aws.yml`.

Fonte: elaborado pelos autores (2026).

Figura 2 – Stack dre-dev criada pelo CloudFormation



Fonte: elaborado pelos autores (2026).

4 CI/CD COM GITHUB ACTIONS

4.1 INTEGRAÇÃO CONTÍNUA (CI)

Quadro 3 – Integração contínua (CI)

Quando roda	• Em todo push e pull request.
O que verifica	• Sintaxe de todos os arquivos PHP e testes unitários (helpers, Logger e estratégias do DRE). • Integração com um MySQL 8 real: migração do banco e smoke test HTTP (health, login, rotas e bloqueio de `.env`, código-fonte e dump SQL). • Build da imagem Docker, ShellCheck dos scripts, validação do template com `cfn-lint` e varredura de segredos com o gitleaks.
Benefícios	• Um erro é barrado antes de chegar na AWS, e um commit com senha ou chave é recusado pela pipeline.

Fonte: elaborado pelos autores (2026).

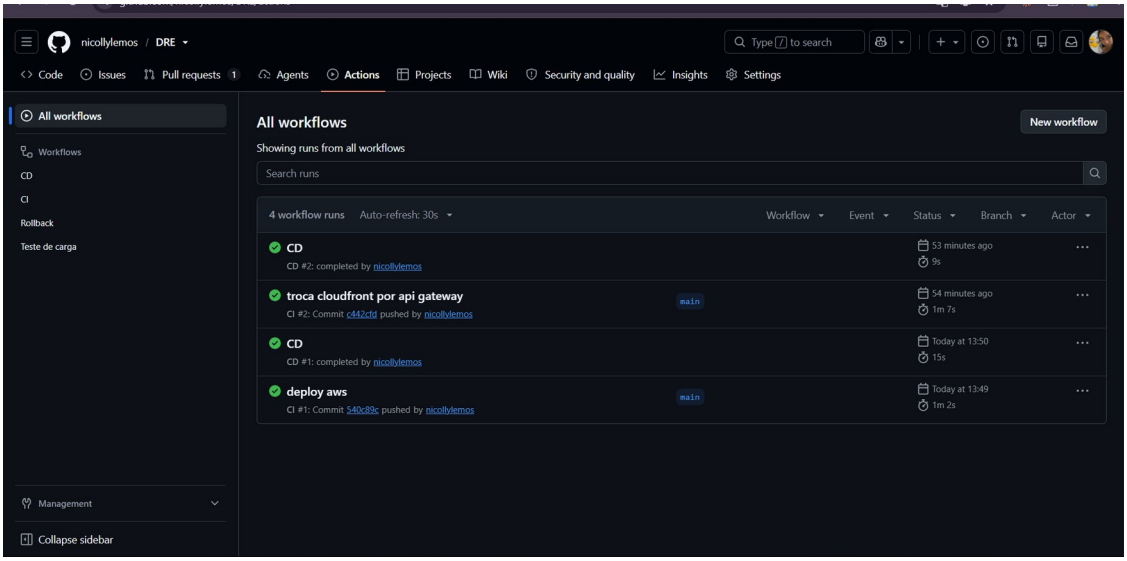
4.2 ENTREGA CONTÍNUA (CD)

Quadro 4 – Entrega contínua (CD)

Como funciona	• Quando o CI passa na `main`, o workflow de CD entra na EC2 por SSH e roda `deploy.sh` com o commit exato. • O script monta o `.env` a partir do Secrets Manager, faz o build, troca o container e só confirma a versão se o `/health` ficar UP. Depois o CD testa a URL pública e cria a tag `deploy-dev-…` no GitHub.
Limitação do Academy	• As credenciais AWS do laboratório expiram a cada sessão e não é possível criar usuário IAM nem OIDC, por isso o deploy entra por SSH com a chave do laboratório.

Fonte: elaborado pelos autores (2026).

Figura 3 – CI e CD executados no GitHub Actions



Fonte: elaborado pelos autores (2026).

5 SEGURANÇA

Quadro 5 – Segurança

Secrets	• O `.env` não vai para o Git. Na AWS a senha do RDS é gerada pelo Secrets Manager, entregue ao banco pelo CloudFormation e lida pela EC2 no deploy; o arquivo fica com permissão 600. • No GitHub ficam só o IP e a chave SSH, como secrets criptografados.
Rede	• O RDS não tem IP público e só aceita conexões da EC2. Discos EBS e RDS são criptografados e a EC2 exige IMDSv2. • O API Gateway envia em toda requisição o cabeçalho `X-Origin-Verify` com um segredo gerado no Secrets Manager. Quem tenta acessar o IP da EC2 direto recebe 403 — o que aparece de verdade nos logs (Figura 5).
Aplicação	• Continuam valendo o bcrypt, o CSRF, o bloqueio por tentativas de login e os cookies seguros, agora reconhecendo o HTTPS do API Gateway. O IP real do usuário chega num cabeçalho que o API Gateway sobrescreve, então não dá para forjá-lo.

Fonte: elaborado pelos autores (2026).

5.1 IAM E LEAST PRIVILEGE

No laboratório a EC2 usa a `LabRole` (pelo `LabInstanceProfile`), porque não podemos criar roles. Em produção ela teria uma role própria com só o necessário — a policy pronta está em `docs/aws/iam-producao.json`:

Quadro 6 – Permissões mínimas da EC2 em produção

Ler a senha	• `secretsmanager:GetSecretValue` apenas nos segredos `dre-prod-*`.

Enviar logs	• `logs:CreateLogStream` e `logs:PutLogEvents` apenas no log group `/dre/prod/app`.
Backup	• `s3:PutObject`, `s3:GetObject` e `s3:ListBucket` apenas no bucket de backup.

Fonte: elaborado pelos autores (2026).

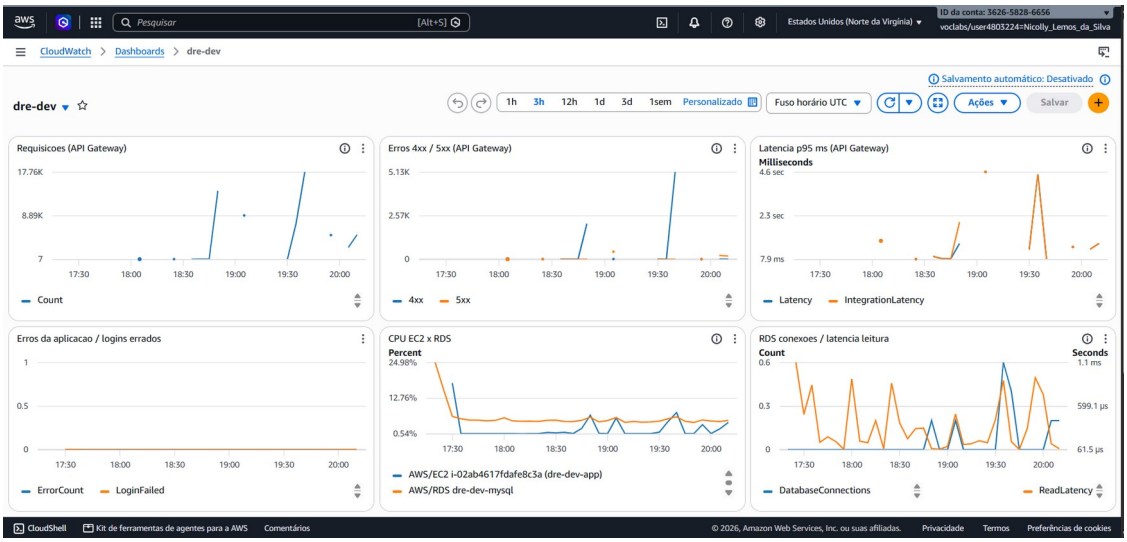
6 LOGS E MONITORAMENTO

Quadro 7 – Logs e monitoramento

Logs	• A aplicação grava uma linha JSON por evento com nível INFO (operações como login e lançamentos), WARN (login errado, acesso negado) ou ERROR (falha de banco, exceção). • O Docker envia os logs para o CloudWatch Logs (`/dre/dev/app`, 30 dias). Senha, token, cookie e CSRF são mascarados antes de gravar.
Health check	• `GET /health` testa o banco e responde `{"status":"UP","version":"3.3.0"}` (ou 503 com DOWN). É usado pelo Docker, pelo deploy, pelo CD e pelo teste de carga.
Métricas e alarmes	• Dashboard `dre-dev` com requisições, erros 4xx/5xx e latência do API Gateway, CPU da EC2 e do RDS, conexões do banco e os últimos WARN/ERROR. • Oito alarmes com aviso por e-mail: erros da aplicação, possível força bruta no login, CPU da EC2 e do RDS, status da EC2, disco do RDS, 5xx e latência do API Gateway.

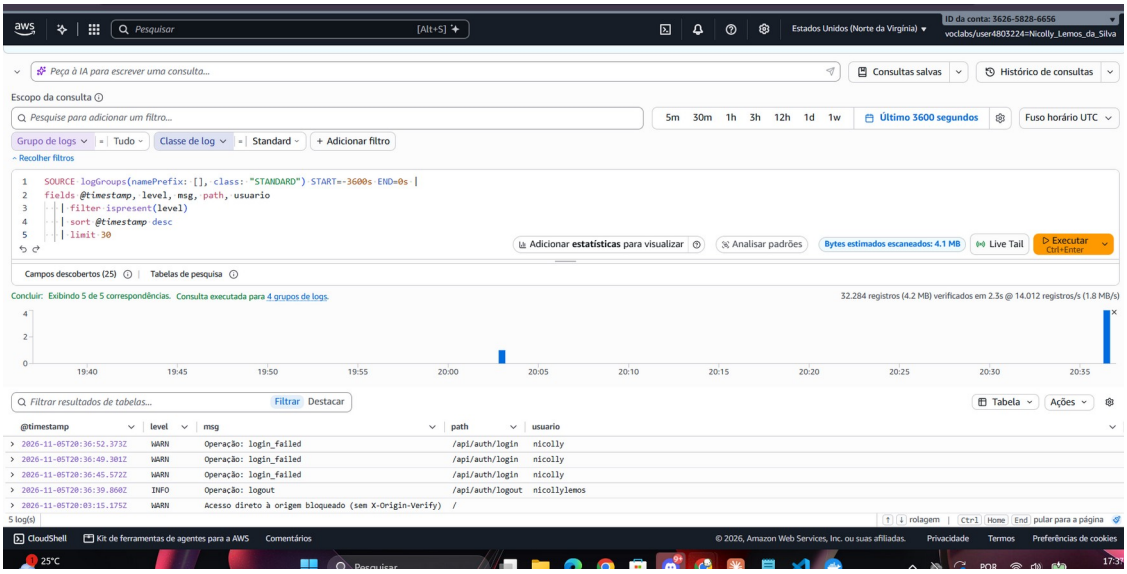
Fonte: elaborado pelos autores (2026).

Figura 4 – Dashboard dre-dev no CloudWatch



Fonte: elaborado pelos autores (2026).

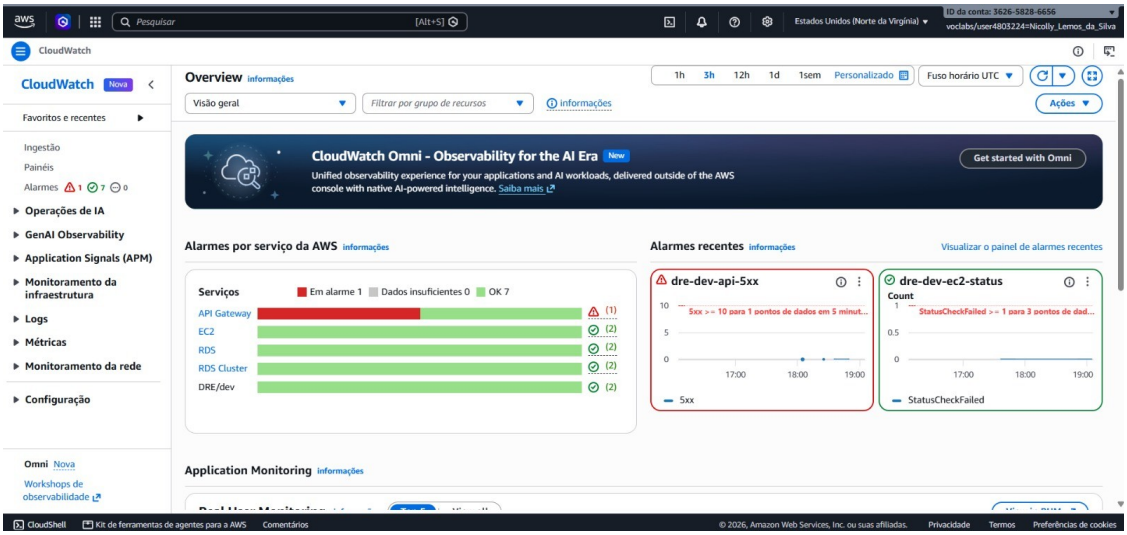
Figura 5 – Logs no CloudWatch Logs Insights: INFO, WARN e o bloqueio de acesso direto



Fonte: elaborado pelos autores (2026).

O monitoramento funcionou sem ninguém olhar: durante um teste de carga houve um deploy, o site ficou alguns segundos fora e o alarme `dre-dev-api-5xx` disparou sozinho (Figura 6).

Figura 6 – Alarme de erros 5xx disparado pelo CloudWatch



Fonte: elaborado pelos autores (2026).

7 BACKUP E RECUPERAÇÃO

Quadro 8 – Backup e recuperação

Banco	• Backup automático do RDS com restore em ponto no tempo, guardado por 7 dias. • Dump diário do MySQL pelo `backup.sh` (cron às 03h30), enviado ao S3 e guardado por 30 dias.

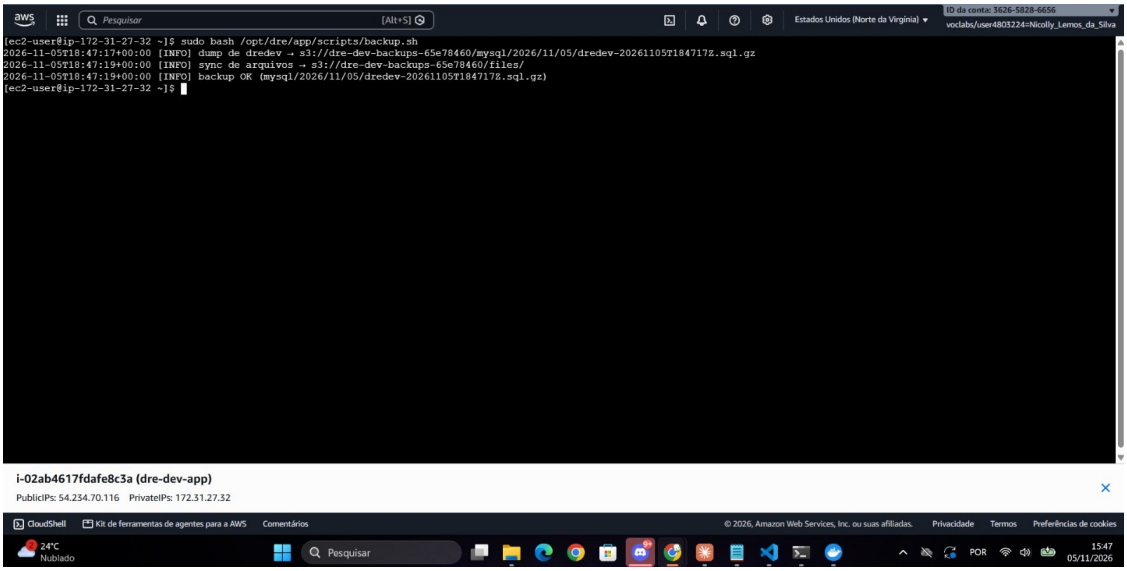
Arquivos	• Uploads, comprovantes, notas e contratos sincronizados todo dia com o S3, que é versionado.
Se perder dados	• Registro apagado ou errado: `restore.sh db` volta o último dump. • Banco perdido: restore em ponto no tempo do RDS. • EC2 perdida: recriar pelo template e rodar `restore.sh files`.

Fonte: elaborado pelos autores (2026).

7.1 RESTORE TESTADO

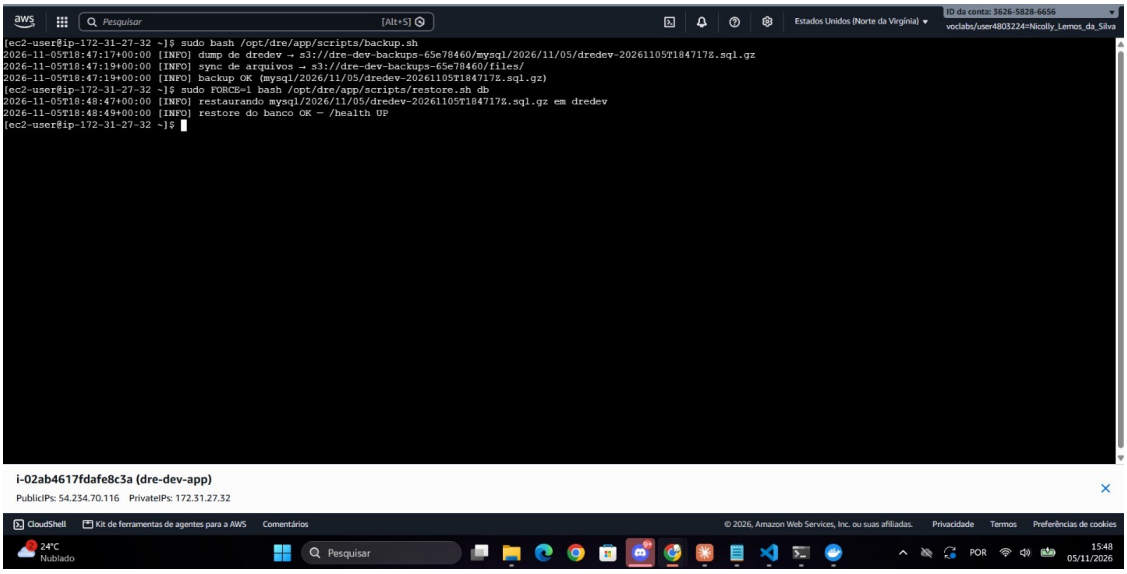
Fizemos o roteiro completo em 05/11/2026: cadastramos um cliente de teste, rodamos o backup, apagamos o cliente pela tela (falha simulada), restauramos o banco e conferimos que o cliente voltou com o sistema operando normalmente. Os dados pessoais do cliente de teste foram borrados nos prints.

Figura 7 – Backup do banco e dos arquivos enviado ao S3



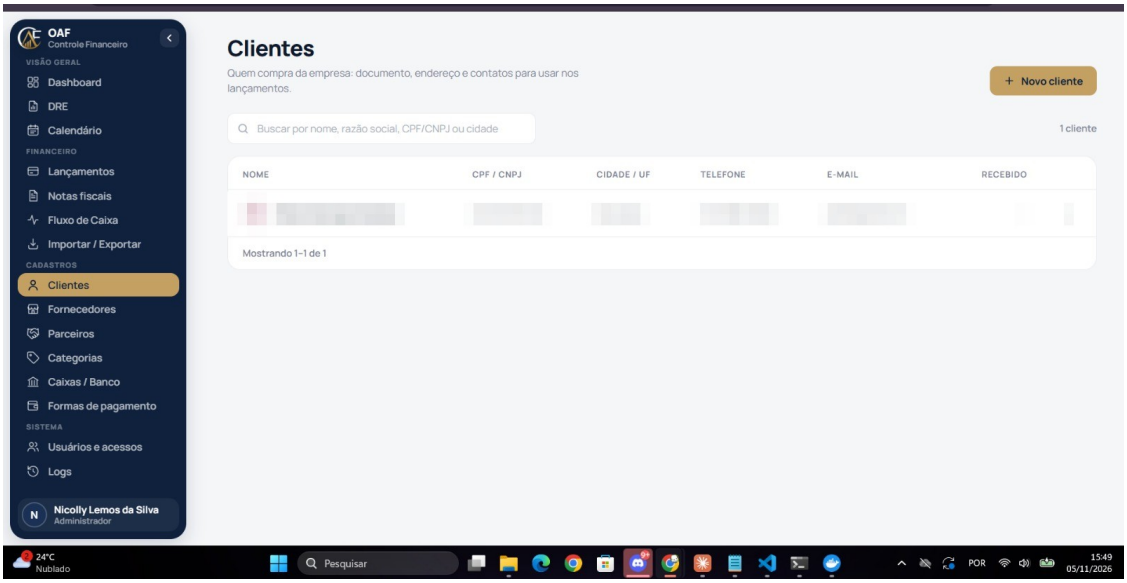
Fonte: elaborado pelos autores (2026).

Figura 8 – Restore do banco com o health check UP



Fonte: elaborado pelos autores (2026).

Figura 9 – Cliente apagado de volta depois do restore



Fonte: elaborado pelos autores (2026).

8 AMBIENTES

O código não tem nada fixo: banco, ambiente e segredos vêm de variáveis de ambiente. O mesmo template sobe DEV, HML ou PROD trocando só o parâmetro `Environment`, e os nomes mudam junto (`dre-dev-mysql`, `dre-hml-mysql`, `dre-prod-mysql`). Em PROD o template já liga sozinho o RDS Multi-AZ e a proteção contra exclusão. No laboratório mantivemos só o DEV.

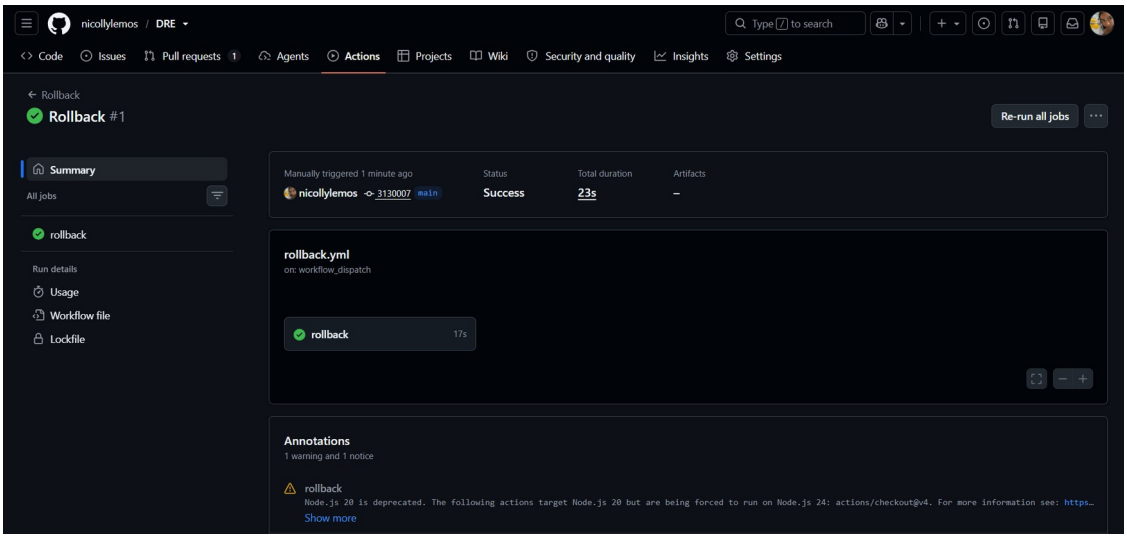
9 ROLLBACK

Quadro 9 – Rollback

Versão nova não sobe	• Automático: o `deploy.sh` só troca a versão se o `/health` ficar UP; senão volta sozinho para a anterior.
Subiu com bug	• Workflow **Rollback** ou `rollback.sh`: reativa a imagem anterior sem rebuild, em segundos. As cinco últimas imagens ficam guardadas.
Versão específica	• Rollback informando uma tag `deploy-dev-…` ou de versão; ele refaz o build daquela versão.
Infraestrutura	• Se uma atualização da stack falha, o CloudFormation desfaz sozinho.

Fonte: elaborado pelos autores (2026).

Figura 10 – Rollback executado pelo GitHub Actions



Fonte: elaborado pelos autores (2026).

10 ESCALABILIDADE, TESTE DE CARGA E CUSTOS

10.1 CRESCIMENTO DA APLICAÇÃO

Quadro 10 – Comportamento esperado com o aumento de usuários

Usuários	Comportamento	O que muda
100	Folgado.	Nada.
1.000	A EC2 e o RDS pequenos ainda dão conta; o throttling do API Gateway segura rajadas.	Instâncias um tamanho maior se o alarme de CPU tocar.
10.000	Gargalo: uma EC2 só, que também é ponto único de falha. Sessões e uploads no disco local impedem ter várias.	Uploads no S3, sessões no banco ou no Redis, ALB + Auto Scaling, RDS Multi-AZ.
100.000	O banco vira o gargalo nos relatórios pesados.	ECS Fargate, Aurora com réplicas de leitura, cache dos relatórios e WAF.

Fonte: elaborado pelos autores (2026).

Escalam sozinhos hoje o API Gateway, o S3, o CloudWatch, o Secrets Manager e o disco do RDS. A EC2 e a CPU do RDS não.

10.2 TESTE DE CARGA

Usamos o k6 pelo GitHub Actions contra a URL pública. Cada usuário virtual abre o `/health` (que consulta o banco), a tela de login e o CSS.

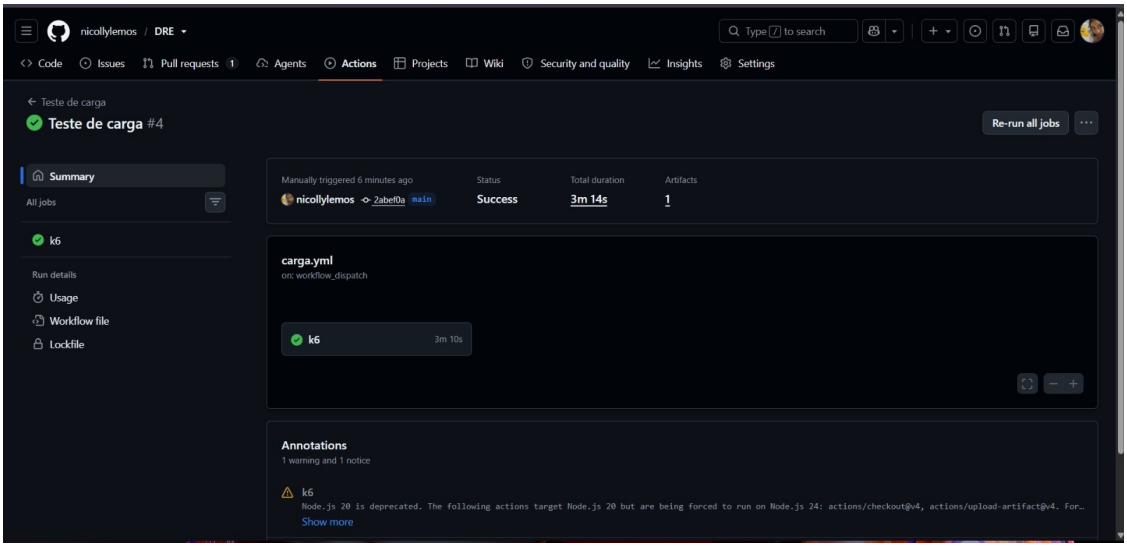
Tabela 1 – Resultados do teste de carga com o k6

Cenário	Usuários	Erros	Mediana	p95	Resultado
Carga (uso normal)	até 30	0%	78 ms	715 ms	Metas atingidas
Estresse (achar o limite)	até 100	0%	76 ms	4,6 s	Vazão travou em ~37 req/s

Fonte: elaborado pelos autores (2026).

No estresse nenhuma requisição falhou, mas a partir de ~37 requisições por segundo elas entraram em fila. O dashboard mostrou que não foi falta de CPU: a EC2 ficou abaixo de 5% e o banco quase sem conexões, enquanto o tempo de espera do API Gateway pela EC2 subiu. O limite está no caminho até a única instância. Também vimos que, com uma EC2 só, todo deploy deixa o site alguns segundos fora; em produção isso se resolve com deploy blue/green atrás de um balanceador.

Figura 11 – Teste de estresse com o k6 no GitHub Actions



Fonte: elaborado pelos autores (2026).

10.3 CUSTOS

Estimamos o custo fora do Academy no AWS Pricing Calculator, em us-east-1 e sob demanda, para um cenário de 10.000 usuários por mês, 1 milhão de requisições e 10 GB de armazenamento.

Tabela 2 – Estimativa de custo mensal no AWS Pricing Calculator

Serviço	Configuração	US$/mês
Amazon EC2	t3.small Linux, uso constante	15,18
Amazon RDS for MySQL	db.t3.micro Multi-AZ, 20 GB gp3	29,42
Amazon API Gateway	HTTP API, 1 milhão de requisições	1,00
Amazon S3	10 GB Standard	0,23
AWS Secrets Manager	2 segredos	0,80
Amazon VPC	1 IPv4 público (Elastic IP)	3,65
Total	**603,36 em 12 meses**	**50,28**

Fonte: elaborado pelos autores (2026).

O serviço mais caro é o **RDS**, porque fica ligado o mês todo e, em Multi-AZ, mantém uma cópia de reserva em outra zona. No ambiente DEV, com o banco em Single-AZ, o total cai para cerca de US$ 40. Estimativa pública: AWS Pricing Calculator.

11 AWS ACADEMY × PRODUÇÃO

Algumas práticas de produção não puderam ser feitas no laboratório. Para cada uma registramos como ficou no Academy e como seria numa conta AWS real.

11.1 CDN E HTTPS

Quadro 11 – CDN e HTTPS: AWS Academy e produção

AWS Academy	• API Gateway com o certificado da própria AWS.
Limitação	• O laboratório nega `cloudfront:CreateDistribution` (erro 403 ao criar a stack) e não há domínio próprio.
Produção	• CloudFront na frente com cache dos estáticos e WAF, liberando no Security Group só os IPs do CloudFront, e domínio no Route 53 com certificado do ACM.

Fonte: elaborado pelos autores (2026).

11.2 IAM

Quadro 12 – IAM: AWS Academy e produção

AWS Academy	• A EC2 usa a LabRole fornecida pelo laboratório.
Limitação	• Não é possível criar Roles e Policies.
Produção	• Uma role por componente com Least Privilege (seção 5.1).

Fonte: elaborado pelos autores (2026).

11.3 DEPLOY

Quadro 13 – Deploy: AWS Academy e produção

AWS Academy	• GitHub Actions entra na EC2 por SSH com a chave do laboratório, com a porta 22 aberta só para chave.
Limitação	• As credenciais AWS expiram a cada sessão; não dá para criar usuário IAM nem OIDC.
Produção	• GitHub Actions com OIDC assumindo uma role de deploy, imagem no ECR e SSM Run Command ou CodeDeploy, sem porta 22.

Fonte: elaborado pelos autores (2026).

11.4 DISPONIBILIDADE

Quadro 14 – Disponibilidade: AWS Academy e produção

AWS Academy	• O laboratório desliga a EC2 e o RDS no fim da sessão; ao reiniciar, o sistema volta sozinho com o mesmo endereço.
Limitação	• Sessão com tempo limitado.
Produção	• Funcionamento 24 horas com balanceador, duas zonas e RDS Multi-AZ.

Fonte: elaborado pelos autores (2026).

11.5 AMBIENTES

Quadro 15 – Ambientes: AWS Academy e produção

AWS Academy	• Só o ambiente DEV.
Limitação	• Orçamento e limites do laboratório.
Produção	• Stacks separadas de DEV, HML e PROD, de preferência em contas diferentes.

Fonte: elaborado pelos autores (2026).

11.6 BACKUP

Quadro 16 – Backup: AWS Academy e produção

AWS Academy	• Backup automático do RDS e script de dump e cópia para o S3.
Limitação	• AWS Backup e cópia para outra região limitados.
Produção	• Plano centralizado no AWS Backup com cópia em outra região para Disaster Recovery.

Fonte: elaborado pelos autores (2026).

11.7 UPLOADS

Quadro 17 – Uploads: AWS Academy e produção

AWS Academy	• Arquivos de até 10 MB pela internet.
Limitação	• Limite fixo de tamanho do API Gateway.
Produção	• CloudFront aceita os 20 MB do sistema, ou upload direto ao S3 com URL pré-assinada.

Fonte: elaborado pelos autores (2026).

12 CONCLUSÃO

Ao fim do projeto conseguimos responder às perguntas propostas na disciplina com o sistema funcionando de verdade:

Quadro 18 – Respostas às perguntas propostas na disciplina

Está funcionando?	• Sim, em HTTPS no API Gateway, sem depender do computador de nenhuma integrante.
Como achamos erros?	• Pelos logs JSON no CloudWatch e pelos alarmes, que avisam por e-mail.
Como monitoramos?	• Pelo dashboard `dre-dev` e pelo `/health`.
Como recuperamos dados?	• Pelo restore do dump no S3 ou pelo restore em ponto no tempo do RDS — testado.
Como publicamos?	• Merge na `main`: o CI testa e o CD publica sozinho.
Como voltamos?	• Rollback automático se a versão nova não sobe, ou manual pelo workflow.
E se crescer?	• Até ~1.000 usuários sem mudanças; acima disso, balanceador com várias instâncias.
Quanto custa?	• Cerca de US$ 40 a 50 por mês.
O que falta para produção?	• CloudFront com domínio, IAM com Least Privilege, deploy por OIDC e ambientes separados.

Fonte: elaborado pelos autores (2026).

Mais do que hospedar o sistema, o trabalho nos fez pensar em como operá-lo: o que acontece quando dá erro, quando um dado some ou quando o número de usuários cresce. Os próprios testes mostraram coisas que não esperávamos, como o gargalo fora da CPU e a queda de alguns segundos a cada deploy, e isso entrou na documentação como próximo passo.

REFERÊNCIAS

AMAZON WEB SERVICES. **AWS CloudFormation**: user guide. Seattle: AWS, 2026. Disponível em: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/. Acesso em: 5 nov. 2026.

AMAZON WEB SERVICES. **Amazon API Gateway**: developer guide. Seattle: AWS, 2026. Disponível em: https://docs.aws.amazon.com/apigateway/latest/developerguide/. Acesso em: 5 nov. 2026.

AMAZON WEB SERVICES. **Amazon EC2**: user guide. Seattle: AWS, 2026. Disponível em: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/. Acesso em: 5 nov. 2026.

AMAZON WEB SERVICES. **Amazon RDS**: user guide. Seattle: AWS, 2026. Disponível em: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/. Acesso em: 5 nov. 2026.

AMAZON WEB SERVICES. **AWS Pricing Calculator**. Seattle: AWS, 2026. Disponível em: https://calculator.aws/. Acesso em: 5 nov. 2026.

GITHUB. **GitHub Actions documentation**. San Francisco: GitHub, 2026. Disponível em: https://docs.github.com/actions. Acesso em: 5 nov. 2026.

GRAFANA LABS. **Grafana k6 documentation**. New York: Grafana Labs, 2026. Disponível em: https://grafana.com/docs/k6/latest/. Acesso em: 5 nov. 2026.